NganyaFlow AI Concept Note

Project purpose and problem

We propose NganyaFlow, an Android research prototype that estimates waiting times at selected Nairobi matatu stages and collects the observations needed to evaluate those estimates. Digital Matatus established a useful route and stop data foundation, while documenting gaps in service frequencies and operating schedules [1]. A static route map therefore cannot by itself tell a commuter how long the next vehicle will take. NganyaFlow addresses this information gap and supports the intent of SDG target 11.2 on accessible, sustainable transport [2]. It will not claim to guarantee a vehicle, a seat, or a reduction in waiting time before evaluation.

Research question and objectives

Can an uncertainty-aware model trained on structured field observations estimate time to the next vehicle more accurately than a historical baseline when rider reporting is sparse and uneven? We will (1) build accessible rider, enumerator and crew workflows; (2) collect and validate stage-level observations; (3) train and compare waiting-time models; and (4) measure prediction quality, interval coverage, usability and failure under reduced reporting. The pilot will cover one agreed corridor and three verified boarding stages. Route identifiers, direction and the exact boarding point must be confirmed in the field.

Proposed AI model and techniques

The primary model will be a **Quantile Regression Forest (QRF)**, a supervised ensemble of decision trees extending Random Forest regression [3,4]. It will predict the median wait and the 10th and 90th conditional percentiles, forming an initial 80% prediction interval. Unlike ordinary Random Forest mean prediction, QRF retains outcome distributions within leaves; the spread of individual tree means is not a valid substitute for this interval. We choose this model as a practical candidate for mixed tabular features and nonlinear interactions in a small pilot, subject to comparison with simpler baselines.

At query time t, the target is the number of minutes until the next observed vehicle arrival for that stage, route and direction. Features will include time of day, day type, stage, route, direction, recent report counts, observation freshness and lagged service summaries available before t. Missingness indicators will distinguish absent observations from genuine zero counts. We will exclude future arrivals, later uploads and end-of-session summaries from features. An arrival does not prove seats are available; occupancy will be shown separately.

Service-frequency inference is a supporting research component. Multiple passenger boardings can refer to one vehicle; inter-report gaps are not vehicle headways. Enumerator records will supply reference vehicle events, enabling event matching and reporting-rate sensitivity experiments. We will only infer frequency from rider reports where the reporting process can be calibrated; otherwise frequency is unavailable. Crew screens will initially show qualified activity summaries rather than unvalidated dispatch instructions. Graph neural networks, contextual bandits and multilingual conversational trip planning are later extensions, outside the pilot commitment.

Data collection and validation

We plan 40-60 enumerator-hours across the three stages, covering peak and off-peak periods and weekdays and weekends. This is a collection target, not an existing dataset or an assurance of statistical power. Enumerators will log arrivals, departures, route, direction, occupancy band and observation pauses using session-local vehicle identifiers. Consenting riders may submit boarding reports. Digital Matatus data will seed route topology only after checking its version, terms and current field validity; it is not live ground truth [1]. No training data or trained model is currently present in the repository.

Training examples will be constructed only within uninterrupted observed windows; queries without a subsequent observed arrival are censored, not assigned zero wait. Whole sessions and days will remain together. Earlier days will support training, subsequent days validation, and the latest complete days a locked test set; an additional held-out-stage check will assess spatial transfer. Hyperparameters such as tree count, depth and minimum leaf size will be chosen on validation data. Preprocessing will be fitted only on training data. Random row splitting would leak closely related observations across sets.

Evaluation and success criteria

QRF will be compared with a training-only stage/route/time-band median and a last-known-headway baseline with explicit fallback rules. We will report median prediction MAE in minutes, quantile loss, empirical 80% interval coverage and interval width, broken down by stage and peak period. Session-level resampling will quantify uncertainty. Subsampling at several reporting rates and biased removal of reports will test robustness. A candidate will progress only if it improves held-out MAE over the strongest baseline without unacceptable undercoverage; thresholds and the minimum evidence needed to display estimates will be fixed on validation data before testing. Until then, the interface will clearly use demo values or display no estimate.

Usability sessions will measure completion of stage selection and boarding reporting, task time, errors and understanding of uncertainty. Any actual waiting-time benefit needs a separate, consented comparative field study. Prototype usability or offline simulation alone will not establish causal transport improvements.

System design and responsible deployment

Kotlin and Jetpack Compose with Material 3 will provide the Android interface. ViewModels expose UI state; repositories manage Room storage and a durable synchronization queue. A Python service will validate reports and serve versioned model outputs. Local reads and queued writes will preserve core tasks during outages [5]. Responses will include the estimate, evidence status, observation time and expiry. The app will hide expired estimates, distinguish queued from synchronized reports, support manual stage selection and avoid background location collection. Research consent will be separate and optional; linked reports will be described as pseudonymous. Access controls, retention and deletion procedures must be approved before field collection.

Work plan and expected outputs

Weeks 1-2: confirm scope, permissions, instruments and UI; weeks 3-4: collect observations and build offline workflows; weeks 5-6: train models and calibrate display rules; weeks 7-8: run locked evaluation, usability testing and documentation. Deliverables are the Android prototype, reproducible model pipeline, versioned data dictionary, evaluation report and documented limitations. Inadequate observation coverage triggers further collection or a baseline-only prototype, rather than unsupported AI claims.

References

1. Digital Matatus. (n.d.). Digital Matatus: Developing Open Data for Informal Transit Systems. https://www.digitalmatatus.com/ (accessed 13 October 2026).

2. United Nations Department of Economic and Social Affairs. (n.d.). Goal 11, target 11.2. https://sdgs.un.org/goals/goal11 (accessed 13 October 2026).

3. Breiman, L. (2001). Random forests. Machine Learning, 45, 5-32. https://doi.org/10.1023/A:1010933404324

4. Meinshausen, N. (2006). Quantile regression forests. Journal of Machine Learning Research, 7(35), 983-999. https://www.jmlr.org/papers/v7/meinshausen06a.html

5. Google. (n.d.). Build an offline-first app. Android Developers. https://developer.android.com/topic/architecture/data-layer/offline-first (accessed 13 October 2026).